

UNIVERSIDAD TÉCNICA FEDERICO SANTA MARÍA

Sistemas de información para la gestión – ICN-292

Laboratorio 3

Automatización de procesos en n8n

Segundo Semestre 2026

Gonzalo Orellana

Profesor: José Miguel González

Fecha de entrega: 23 de septiembre de 2026

Índice

Resumen Ejecutivo.....	3
Desarrollo.....	4
Parte A.....	4
Tabla A 15 solicitudes	4
Parte B.....	4
Imagen mensaje obtenido al ejecutar el Flujo	5
Parte C.....	6
1. Configuración y Estrategias On Error	6
2. Explicación Teórica de Error Workflow	7
3. Pruebas de Casos Límite	8
4. Experimento de Error de Tipo	8
Conclusiones	10
Anexos.....	11
Prueba 1	11
.....	11
Prueba 2	12
.....	12
Prueba 3	13
Punto 3 Parte C	14
.....	14
Punto 4 Parte C	16
.....	16

Resumen Ejecutivo

El presente informe trata sobre el desarrollo del Laboratorio 3 de SIG, el cual se realizó usando como base el identificador único $S = 345$, obteniendo así un Umbral de monto $U = 75.000$ clp y un Plazo máximo $D = 14$ días. Para este laboratorio, se diseñó, implementó y probó un prototipo funcional de arquitectura de integración en la plataforma n8n para la empresa AndesHogar SpA. El sistema automatiza el proceso de triage de solicitudes de devolución, integrando un webhook de entrada, una compuerta de reglas condicionales (switch), un enriquecimiento de datos con indicadores económicos externos (UF desde `mindicador.cl`) y un sistema programado de consolidación diaria.

Como principales conclusiones, se llegó a que el motor de reglas procesó el lote de control de 15 solicitudes, logrando un 33.3% de aprobación directa (exactamente 5 casos), derivando un 40% a revisión manual (6 casos por umbral o fallas) y un 26.7% a rechazo (4 casos por plazo o daño). Además, se identificaron vulnerabilidades en la compuerta Fallback que permitían que algunas solicitudes vacías pasaran como aprobadas, así como fallas en comparaciones de cadenas de texto y falta de validación contra catálogo de SKUs.

Para validar las evidencias de verificación, se presenta el URL del repositorio Github que contiene todos los workflows exportados.

<https://github.com/gorellanap/ICN292-Orellana-Gonzalo>

Desarrollo

Parte A

Se construyó un flujo en n8n de 8 nodos que procesa peticiones HTTP POST. Las solicitudes evalúan ordenadamente sus atributos (monto, dias_desde_compra, estado_producto, id_solicitud) mediante la función `.toNumber()` en expresiones numéricas para evitar errores de tipo. Luego de armar el flujo de los 8 nodos, con sus reglas y demás, era necesario crear un segundo workflow emisor con un Trigger manual y un HTTP Request en POST hacia la URL del webhook principal, con el cuerpo en JSON. Ahora sí, luego de tener ambos flujos creados y funcionales, se obtuvo la siguiente tabla, que muestra las 15 solicitudes procesadas del archivo descargado de aula solicitudes.json

Tabla A 15 solicitudes

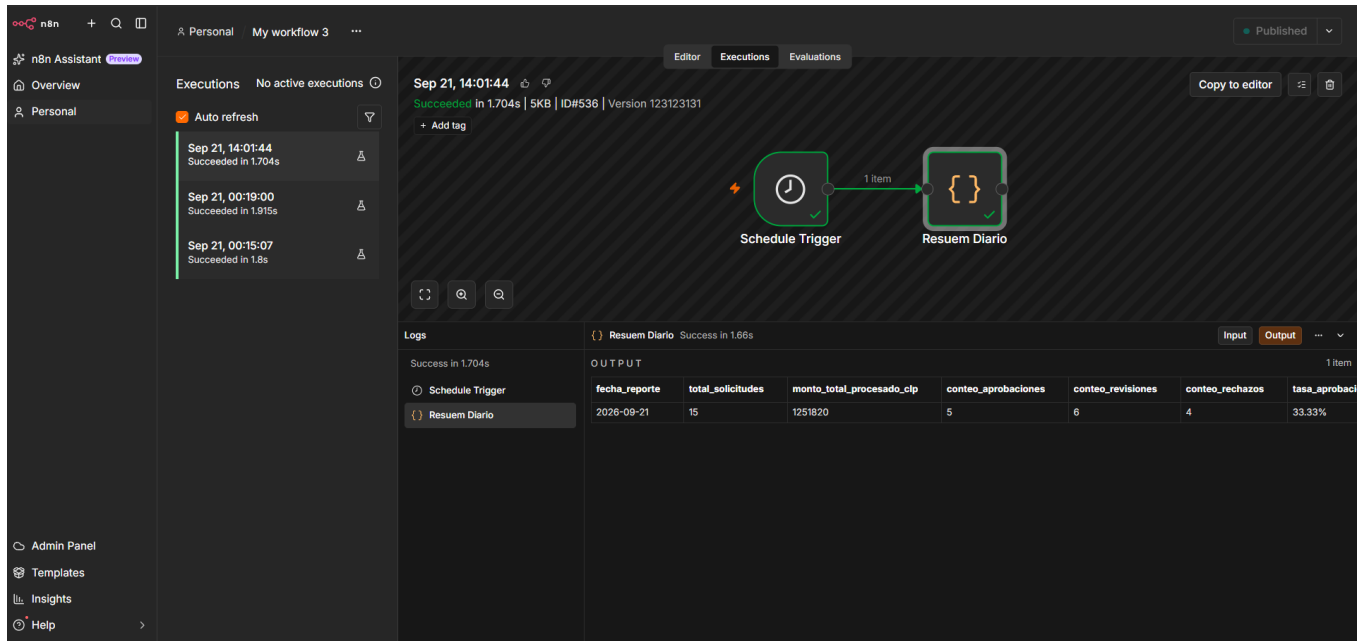
id_solicitud	ruta	motivo	monto_clp	umbral_U	plazo_D
DEV-1115	APROBACION	Solicitud cumple con las políticas de devolucion directa	19990	75000	14
DEV-1116	APROBACION	Solicitud cumple con las políticas de devolucion directa	69980	75000	14
DEV-1117	REVISION	Excede el umbral de \$75.000 o el producto presenta fallas	24990	75000	14
DEV-1118	REVISION	Excede el umbral de \$75.000 o el producto presenta fallas	79990	75000	14
DEV-1119	RECHAZO	Excede el plazo maximo de 14 dias o el producto esta danado por uso	45990	75000	14
DEV-1120	REVISION	Excede el umbral de \$75.000 o el producto presenta fallas	89990	75000	14
DEV-1121	REVISION	Excede el umbral de \$75.000 o el producto presenta fallas	289990	75000	14
DEV-1122	APROBACION	Solicitud cumple con las políticas de devolucion directa	45990	75000	14
DEV-1123	RECHAZO	Excede el plazo maximo de 14 dias o el producto esta danado por uso	39980	75000	14
DEV-1124	APROBACION	Solicitud cumple con las políticas de devolucion directa	49980	75000	14
DEV-1125	RECHAZO	Excede el plazo maximo de 14 dias o el producto esta danado por uso	34990	75000	14
DEV-1126	REVISION	Excede el umbral de \$75.000 o el producto presenta fallas	79990	75000	14
DEV-C01	APROBACION	Solicitud cumple con las políticas de devolucion directa	9990	75000	14
DEV-C02	REVISION	Excede el umbral de \$75.000 o el producto presenta fallas	289990	75000	14
DEV-C03	RECHAZO	Excede el plazo maximo de 14 dias o el producto esta danado por uso	79990	75000	14

Parte B

En esta parte se creó un tercer workflow aparte, el cual se compone de un Schedule Trigger, el cual se pedía que leyera el registro, consolide el día con el nodo Summarize y envíe un solo

mensaje; con solicitudes por ruta, monto total por ruta, monto total del día y tasa de aprobación automática. Importante hay que declarar que la frecuencia elegida del cronograma, se configuró el nodo Schedule Trigger a las 20:00 hrs diariamente

Imagen mensaje obtenido al ejecutar el Flujo



The screenshot displays the n8n interface with a workflow named 'Resuem Diario' published. The workflow is triggered by a 'Schedule Trigger' node and leads to a 'Resuem Diario' node. The execution log for the 'Resuem Diario' node shows the following output:

fecha_reporte	total_solicitudes	monto_total_procesado_clp	conteo_aprobaciones	conteo_revisiones	conteo_rechazos	tasa_aprobacion
2026-09-21	15	1251820	5	6	4	33.33%

De la tabla se identifican las siguientes métricas:

- Total_solicitudes: 15
- monto_total_procesado_clp: 1.251.820
- conteo_aprobaciones: 5
- conteo_revisiones: 6
- conteo_rechazos: 4
- tasa_aprobacion_directa: 33.33%

Además, se pedía explicar que ocurre si un día no se registran solicitudes. Si durante una jornada no ingresan solicitudes al sistema, la consulta o arreglo de entrada retornará una lista vacía de 0 registros. El nodo de procesamiento operará sobre 0 registros, entregando contadores en 0, monto total en 0 y tasa de aprobación en 0% sin arrojar errores de ejecución en n8n. Para evitar

el envío de reportes vacíos, se puede incorporar una compuerta condicional (If) que evalúe si `total_solicitudes > 0` antes de emitir la notificación final.

Parte C

Esta parte era la más extensa del Laboratorio, donde primero se pedía documentar tres pruebas inválidas distintas, cada una con: la carga enviada, el comportamiento esperado, el observado y la captura de la ejecución, donde al menos una debía hacer fallar un nodo.

Tabla con las Pruebas 1,2,3 y demás

Prueba	Carga Enviada	Comportamiento Esperado	Comportamiento Observado
1. Error sintaxis / Error 404	URL errónea / JSON roto	Detención por fallo técnico	El nodo HTTP Request se marca en ROJO
2. Carga útil vacía	JSON vacío ({})	Rechazo (DATOS_INVALIDOS)	Pasa por Fallback hacia APROBACION.
3. Ausencia de parámetro monto	Sin campo "monto" en el JSON de HTTP Request	Rechazo (DATOS_INVALIDOS)	monto = undefined, sale por Fallback a APROBACION

En base a las 3 pruebas realizadas, se puede decir que la Prueba 1 es un fallo no controlado de infraestructura, la Prueba 2 muestra una vulnerabilidad lógica por falta de validación y la Prueba 3 deja en visto un Falso positivo, ya que aprueba sin verificar el monto

Las capturas de las pruebas realizadas, ejecuciones, contenidos de los nodos y más, están presentes en los anexos de este informe.

Siguiendo con la Parte C, se pedían 5 puntos más a desarrollar.

1. Configuración y Estrategias On Error

Se pedía explicar los tres valores de On Error y explicar la razón de cuál se eligió en cada nodo.

Las opciones que el programa n8n presenta son las siguientes:

- Stop Workflow: Cancela la ejecución inmediatamente si el nodo falla.

- Continue (using error output): Agrega una segunda salida al nodo para enviar el error por una rama de contingencia.
- Continue (using fallback): Reemplaza el fallo por un valor por defecto predefinido y continúa la ejecución.

Para la Elección en el flujo principal:

- Webhook y Respond to Webhook: Stop Workflow, para no procesar ni emitir respuestas erróneas
- Switch: Stop Workflow (usando la ruta Fallback activa para capturar datos no clasificados).
- HTTP Request (mindicador.cl): Continue (using fallback), para inyectar un valor por defecto si la API de la UF sufre una caída externa, evitando bloquear la respuesta al cliente.

2. Explicación Teórica de Error Workflow

En este punto se pedía responder qué es un Error Workflow y porqué un flujo programado sin él deja de funcionar en silencio

- Un Error Workflow, es un flujo independiente accionado por un nodo Error Trigger que se dispara automáticamente cuando un flujo principal sufre una falla no controlada.
- Un flujo programado sin Error Workflow deja de funcionar en silencio, porque los flujos programados (como el de la Parte B a las 20:00 hrs) se ejecutan en segundo plano. Si un nodo falla, n8n detiene el proceso y guarda el registro únicamente en los logs internos. Sin un Error Workflow configurado para enviar una alerta, el equipo no se entera de que el reporte diario no se generó.

3. Pruebas de Casos Límite

Para este punto, se pedía realizar 2 casos adicionales, una con monto exactamente igual a U, en este caso 75.000 y otro caso con días_desde_compra igual a D, en este caso 14, y mostrar que el flujo las trata según la regla 6: ambas quedan dentro de lo aceptado.

Para el caso de U, resulta en APROBACION. Demostrando que la regla de revisión es estrictamente $\text{monto} > 75.000$. Al ser $75.000 \leq 75.000$, no se deriva a revisión y se aprueba. Asimismo, para el caso de D, resulta en APROBACION. Demostrando que la regla de rechazo es estrictamente $\text{días} > 14$. El día 14 está dentro del margen legal permitido.

Esto demuestra que las reglas dentro del nodo Switch utilizan desigualdades estrictas, donde en ambos casos ninguna excede los límites y ambas son aprobadas directamente

Las capturas de pantalla de la ejecución realizada, mostrando que pasa por Marcar aprobación está en los anexos.

4. Experimento de Error de Tipo

Para este punto, se pedía cambiar a propósito el tipo de la condición del monto de Number a String, enviando DEV-C01 y mostrar la ruta que se obtiene y porqué. Al cambiar la condición del umbral de monto de Number a String y ejecutar la solicitud DEV-C01 (\$9.990), la solicitud se desvía incorrectamente a REVISION. El nodo Switch colapsa y detiene la ejecución del flujo en rojo. Esto se debe a que, el motor de n8n exige estricta coherencia de tipos cuando se utilizan operadores numéricos. Al enviar el campo monto en formato de texto, n8n no realiza una conversión implícita sobre la marcha y arroja la excepción de sistema Wrong type: '19990' is a string but was expecting a number. Esto demuestra la necesidad operativa de aplicar .toNumber() en las entradas JSON antes de realizar comparaciones de umbrales numéricos en flujos de integración de datos.

Las capturas de pantalla de la prueba realizada se encuentra en la sección de anexos

5. Dos Casos Inresolubles por el Flujo

Para este último punto se pedía enviar una solicitud con “monto”: “veinte mil” y otra con “sku”: “ZZ-999”, en donde la 1era hace fallar un nodo y la 2nda pasa la validación y se clasifica como si

fuera válida. Hablando sobre la 1era solicitud, esta provoca el colapso del nodo Switch al fallar la función de conversión matemática `.toNumber()` sobre texto. Esto ocurre, porque la función de conversión `.toNumber()` del nodo Switch no puede pasar texto en palabras a un entero, colapsando el flujo. Para lograr detectar esto, el flujo necesitaría una etapa previa de validación de entradas mediante expresiones regulares o un nodo Code que convierta las entradas de texto a numérico entero. Por otro lado, con respecto a la 2da solicitud, al ser un código de producto inexistente, el flujo procesa la solicitud y la aprueba si cumple monto y días, debido a que no existe una consulta contra el catálogo maestro o base de datos de la empresa para validar la existencia real del SKU. Esto ocurre porque el flujo solo evalúa condiciones de monto, días y estado físico, careciendo de un nodo que consulte contra el catálogo maestro o base de datos de la empresa. Entonces, lo que el flujo necesitaría integrar sería una llamada a una API interna o consulta SQL a la base de datos de productos para verificar si el SKU existe y está activo antes de clasificar.

Conclusiones

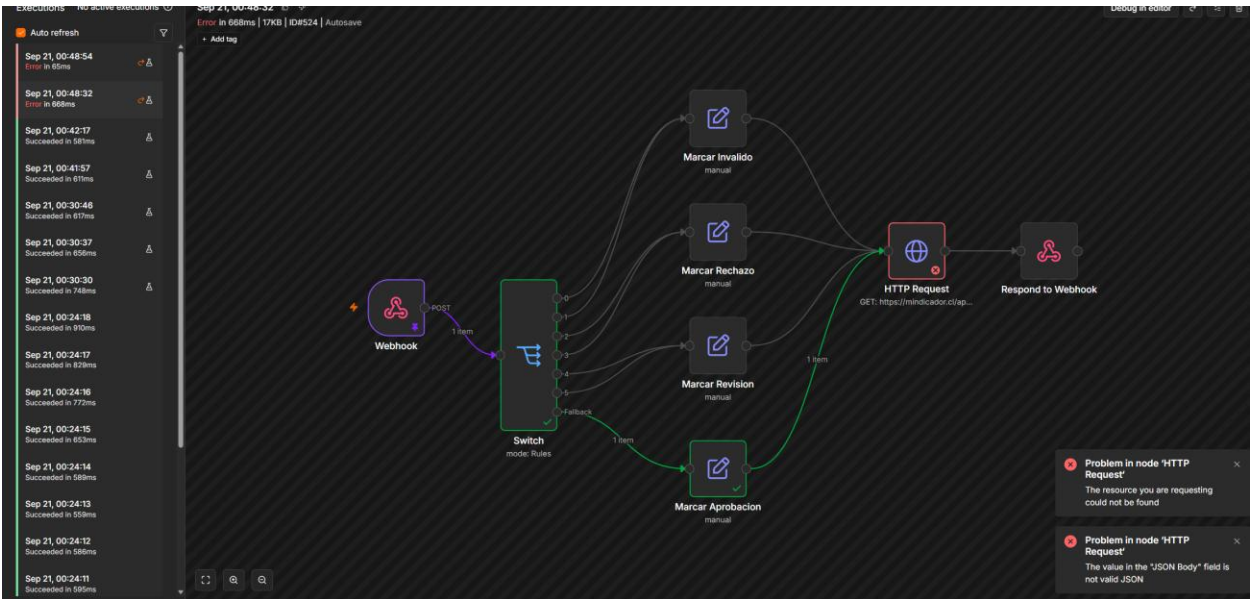
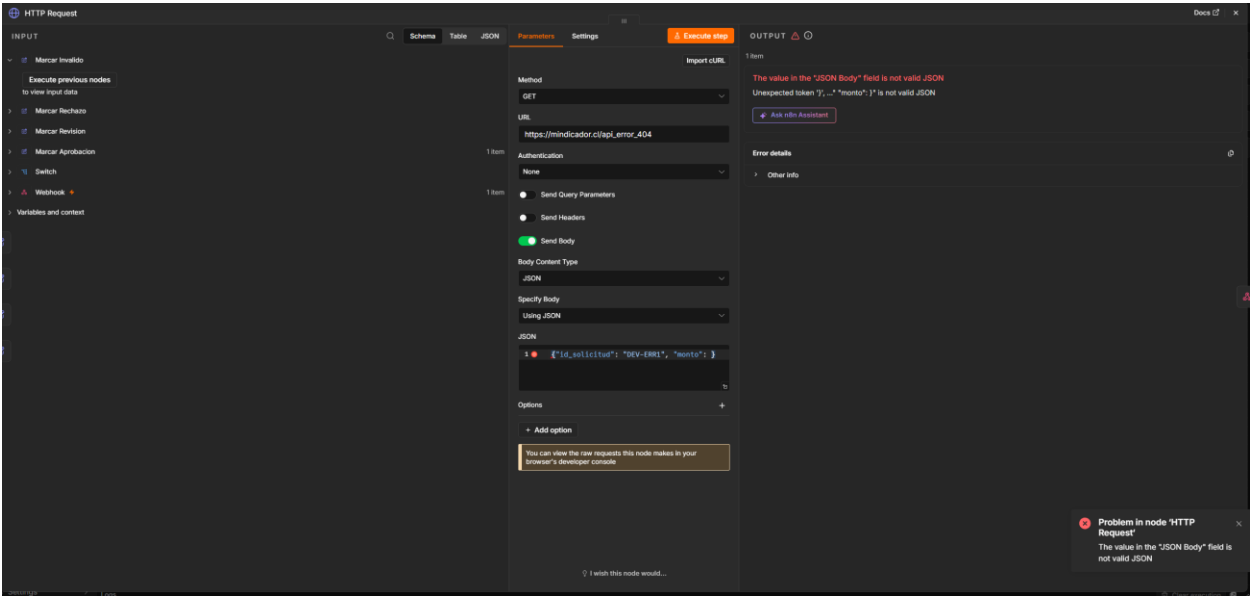
La implementación de la arquitectura de integración en n8n para AndesHogar SpA demostró cómo un motor de reglas de negocio permite procesar y clasificar devoluciones en tiempo real de forma estandarizada. Al evaluar el lote de control de 15 solicitudes con la semilla $S = 345$ ($U = \$75.000$ CLP y $D = 14$ días), el sistema logró responder de forma correcta, otorgando un 33.3% de aprobación automática directa (5 casos), derivando un 40.0% a revisión manual (6 casos) y rechazando un 26.7% (4 casos por estar fuera de plazo o dañados).

Además, los experimentos de la Parte C evidenciaron que la automatización de procesos sin una rigurosa validación de tipos de datos genera graves fallos operativos y de lógica de negocio. Se comprobó que omitir la función de conversión `.toNumber()` provoca el colapso del flujo o desvíos erróneos debido a comparaciones alfabéticas/lexicográficas.

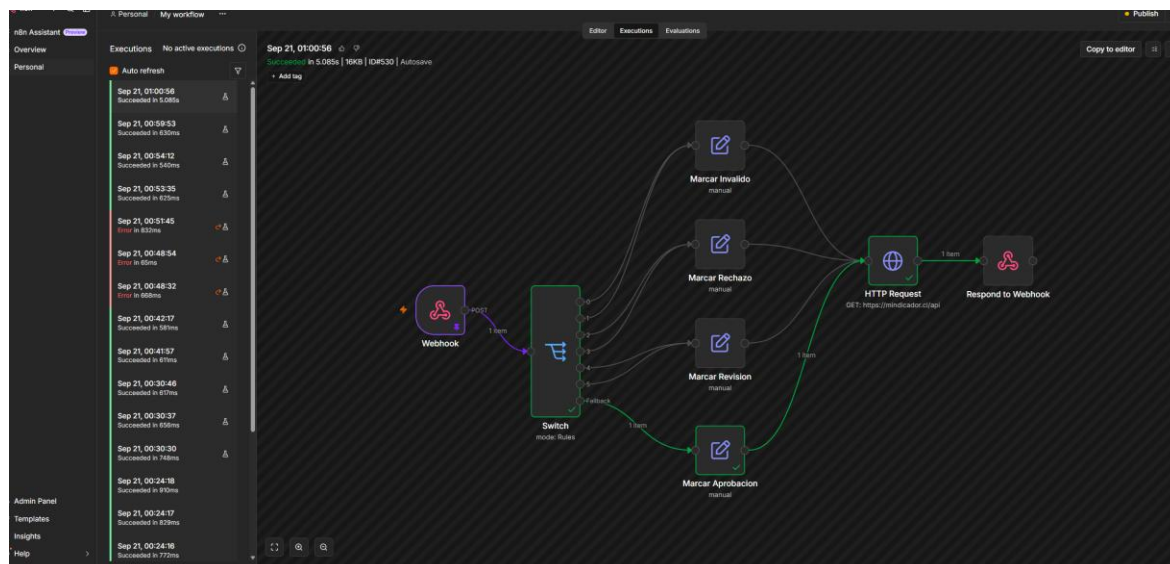
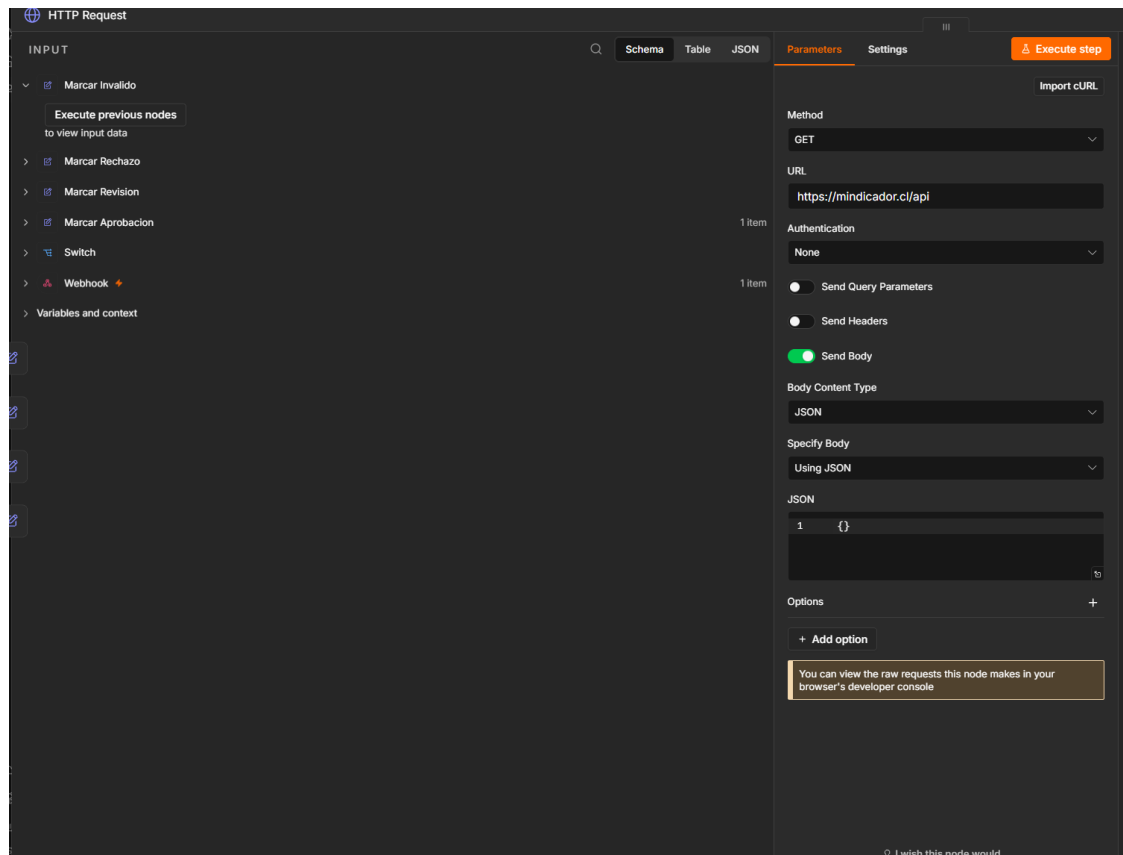
Por último, se identificó que las compuertas de decisión de tipo Fallback representan un riesgo si no cuentan con reglas previas de existencia de objeto, ya que cargas útiles vacías (`{ }`) o sin el campo `monto` derivaron por defecto hacia la ruta de aprobación. Asimismo, la incapacidad del flujo para detectar SKUs inexistentes (como "ZZ-999") o montos en palabras (como "veinte mil") reafirma la regla de la gestión de información, en donde un motor de automatización sin validación contra el catálogo maestro o base de datos de la empresa solo automatiza el error, como se mostró en el punto último de la Parte C

Anexos

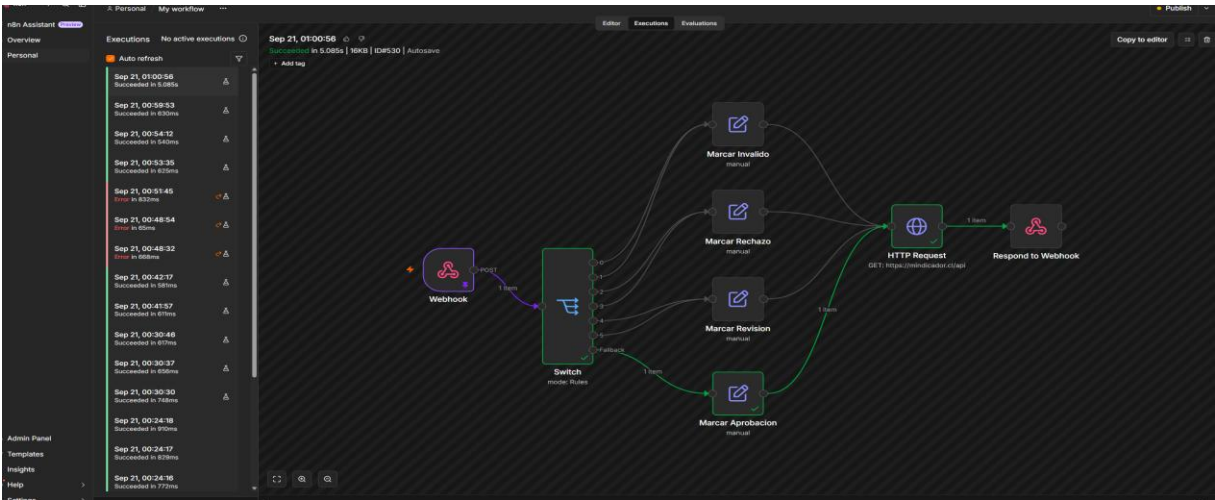
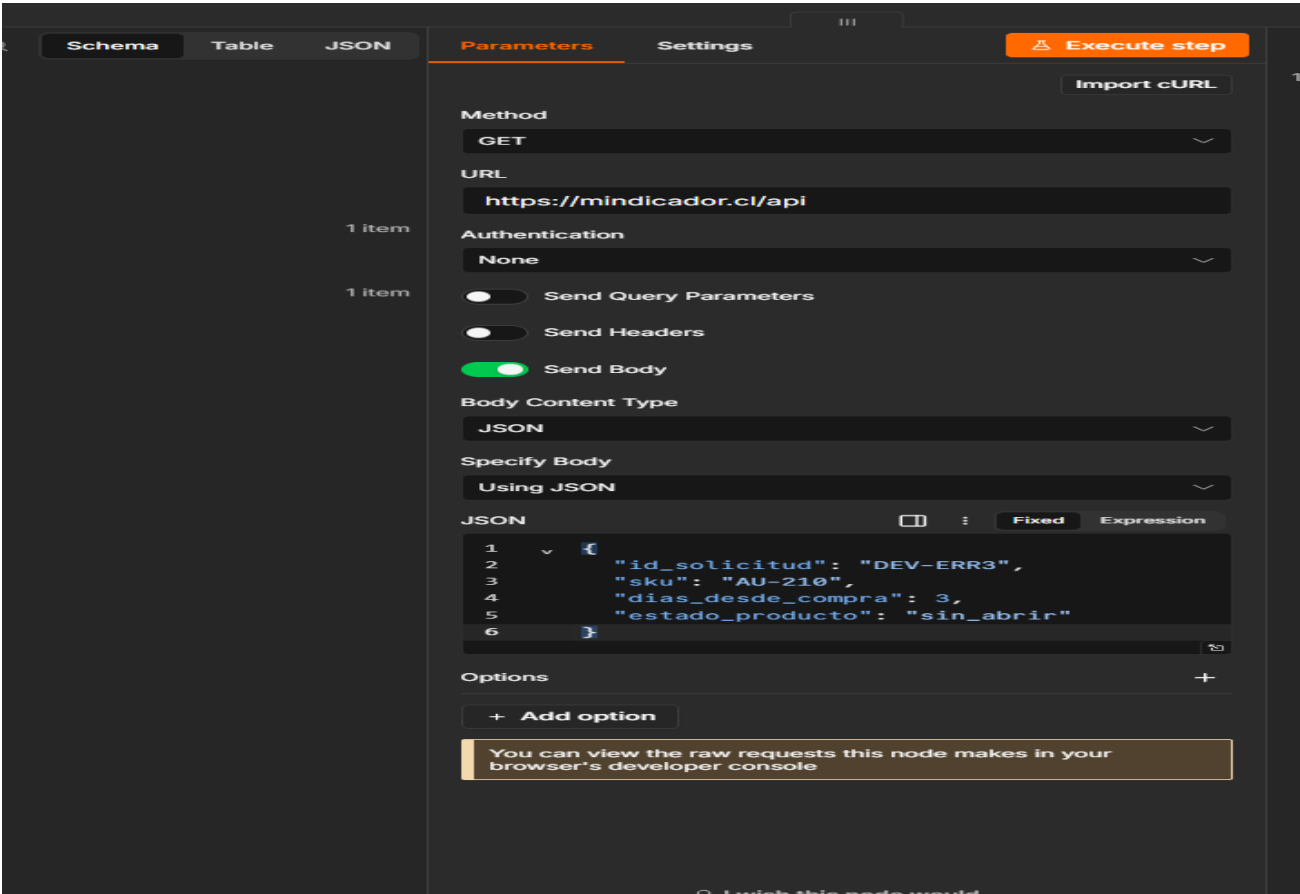
Prueba 1



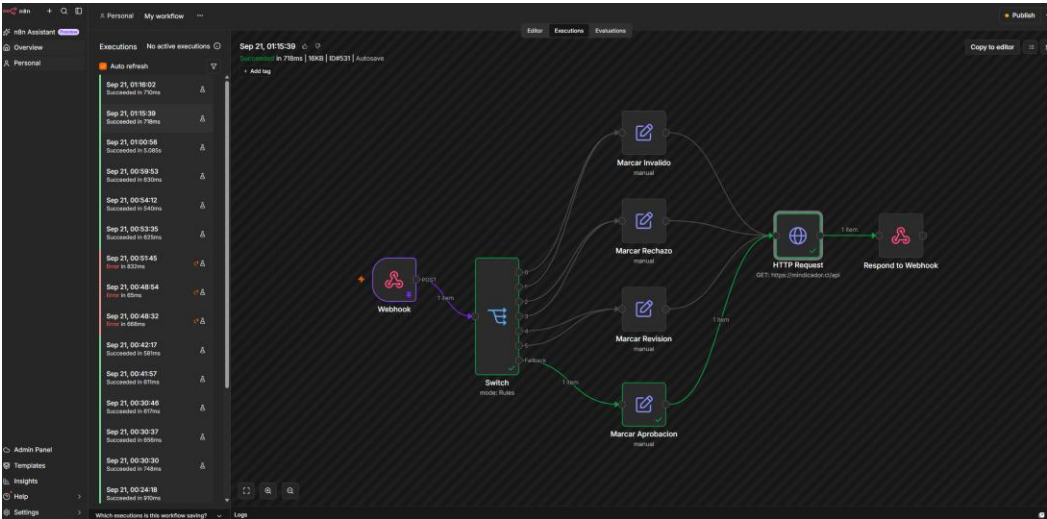
Prueba 2



Prueba 3



Punto 3 Parte C



JSON Parameters Settings

Method: GET

URL: https://mindicador.cl/api

Authentication: None

1 item

1 item

Send Query Parameters

Send Headers

Send Body

Body Content Type: JSON

Specify Body: Using JSON

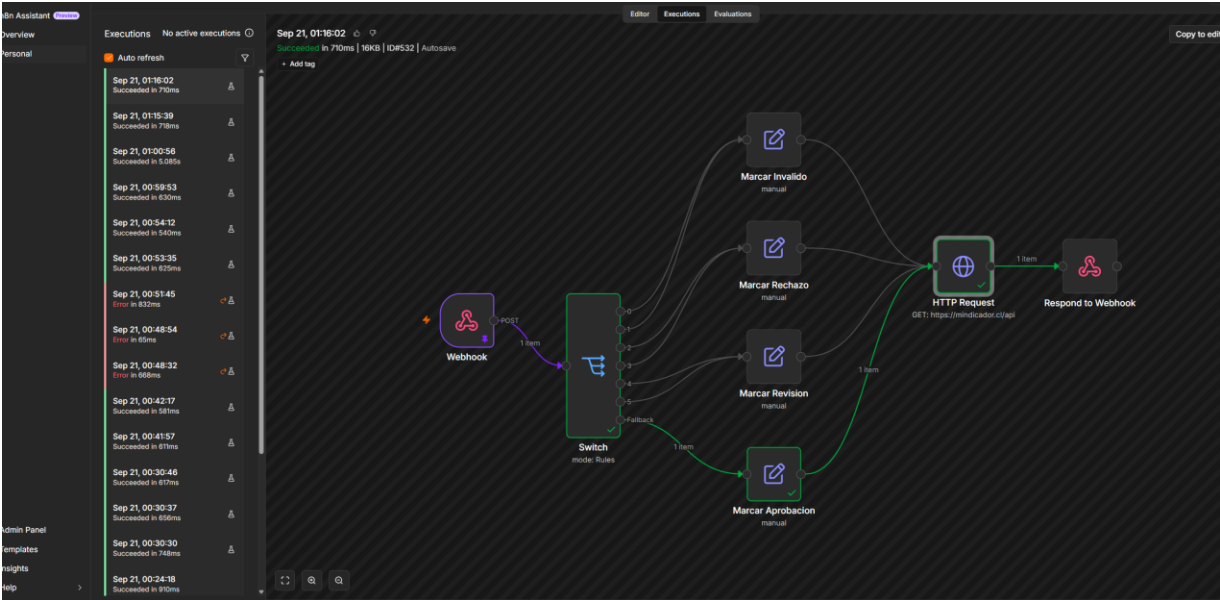
JSON

```
1 {
2   "id_solicitud": "DEV-LIM1",
3   "sku": "AU-210",
4   "monto": "75000",
5   "dias_desde_compra": "5",
6   "estado_producto": "sin_abrir",
7   "email_cliente": "limite1@example.cl"
8 }
```

Options

You can view the raw requests this node makes in your browser's developer console

I wish this node would...



Parameters Settings

Import cURL

Method: GET

URL: https://mindicador.cl/api

Authentication: None

☐ Send Query Parameters

☐ Send Headers

☒ Send Body

Body Content Type: JSON

Specify Body: Using JSON

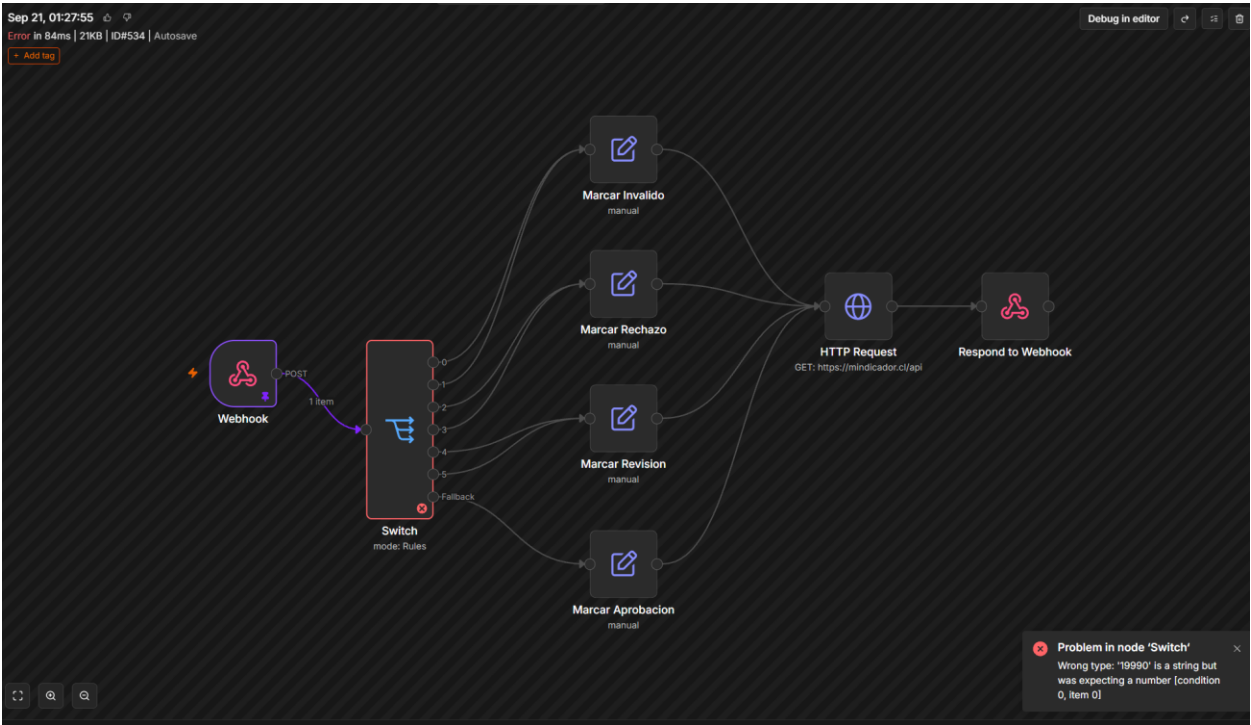
JSON

```
1 {
2   "id_solicitud": "DEV-LIM2",
3   "sku": "AU-210",
4   "monto": "19990",
5   "dias_desde_compra": "14",
6   "estado_producto": "sin_abrir",
7   "email_cliente": "limite2@example.cl"
8 }
```

Options

You can view the raw requests this node makes in your browser's developer console

Punto 4 Parte C



The 'Routing Rule 5' configuration panel is shown. It contains a text input field with the expression `{{ $json.body.monto }}`, a dropdown menu with the text `# is greater than or equal to`, and a text input field with the value `75000`. There is a 'Rename Output' toggle switch at the bottom, which is currently turned off. A warning icon is visible on the right side of the panel.